Solving Problems by Search

Dr Angeliki Koutsoukou-Argyraki

Royal Holloway, University of London
https://dicelab-rhul.org

Credits: Prof Kostas Stathis
Contact: *angeliki.koutsoukouargyraki@rhul.ac.uk*

Ch 3 (§ 3.1 – § 3.3).

Learning outcomes

We shall discuss problem solving and search. At the end of this topic, you should be able to:

- Appreciate why we need programs that solve problems.
- Understand different problem types for search.
- Explain how to formulate search problems.

Outline

1. Solving problems by search
2. Problem types
3. Problem formulation
4. Example problems

Solving problem by search

Interested in AI programs that solve problems by search for us:

- Problem solution can be abstracted as **path** from some (**start**) node to some (**goal**) node in a suitably defined **directed graph**.
- Search algorithms to find (**optimal**) paths.

Example: Romania

On holiday in Romania at *Arad*. Flight leaves tomorrow from *Bucharest*.

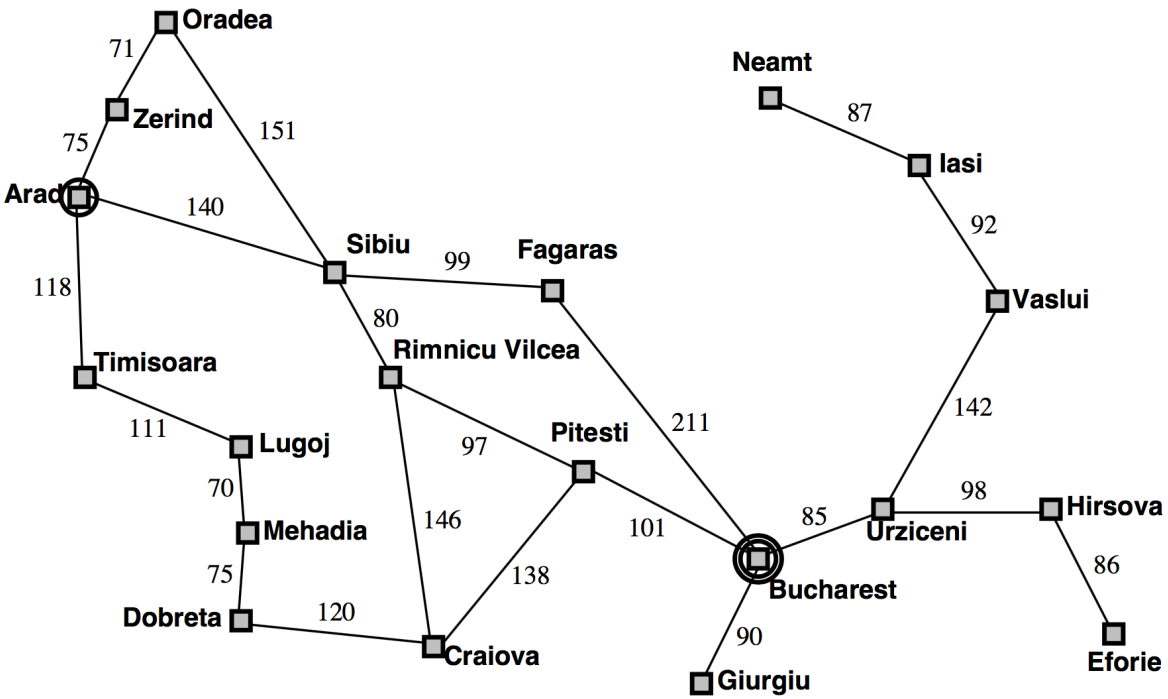
Problem:

- Start: Arad
- Goal: Bucharest

Optimal path:

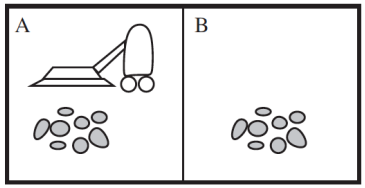
1. shortest; or
2. quickest; or
3. cheapest.

Graph:



Numbers if (1) are distance, (2) are time, (3) are cost/money.

Example: Vacuum World

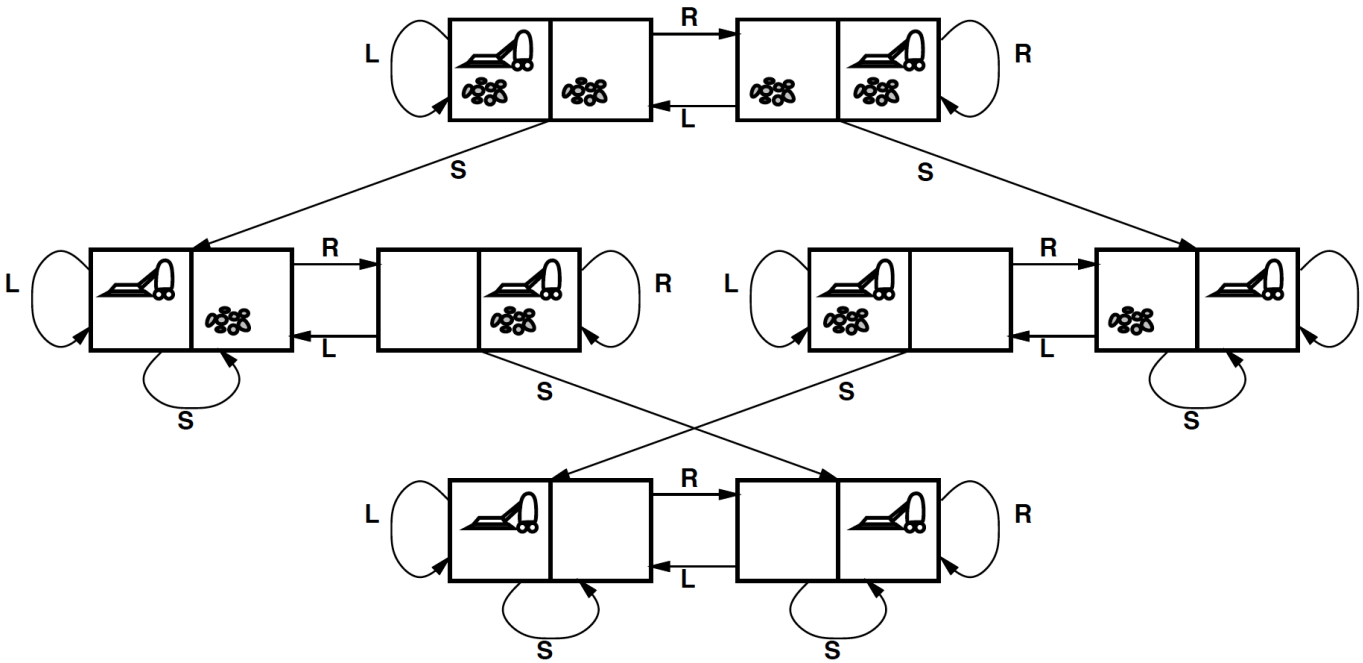


Vacuum cleaner must remove all dirt from a room.

Problem:

- Start: A
- Goal: no dirt anywhere in A, B

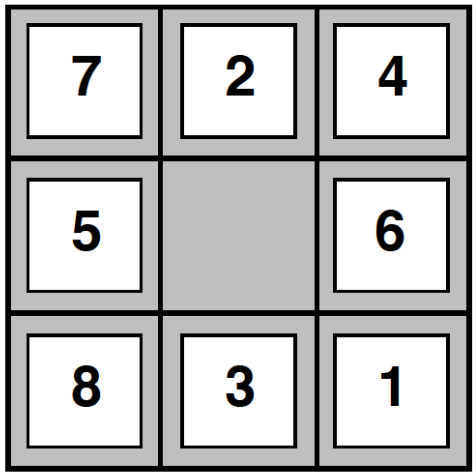
Graph:



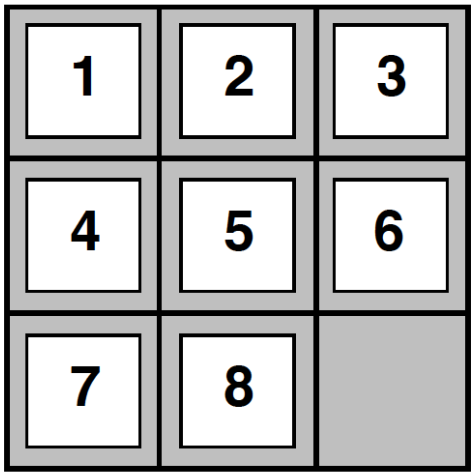
Optimal path:

1. shortest; or
2. quickest; or
3. cheapest.

Example: The 8-puzzle



Start State



Goal State

Optimal path?

Graph?

Exercise for the reader: Try constructing part of the graph, then see book ch. 3, pp. 102-108 (after we cover the Informed Search topic).

Outline

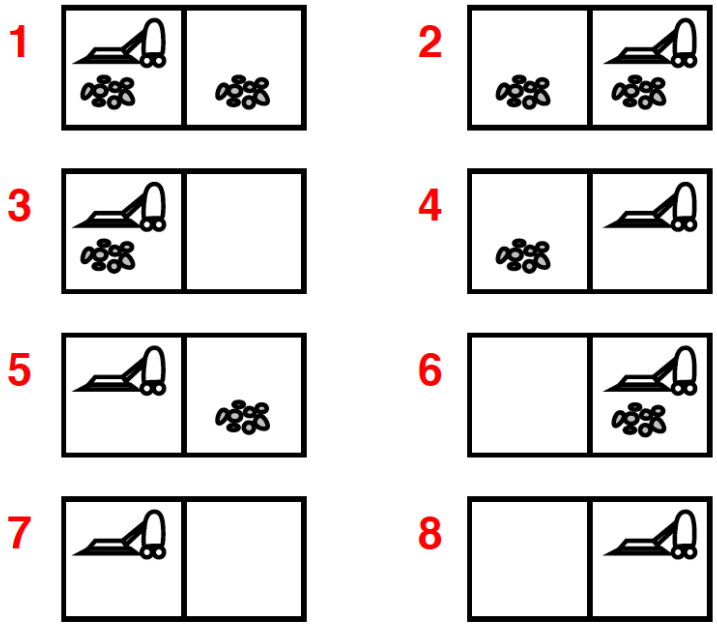
1. Solving problems by search
2. Problem types
3. Problem formulation
4. Example problems

Problem types

- Deterministic, fully observable $\Longrightarrow$ single-state problem
  AI program knows exactly which state it will be in; solution is a sequence.
- Non-observable $\Longrightarrow$ conformant problem
  AI program may have no idea where it is; solution (if any) is a sequence.
- Nondeterministic and/or partially observable $\Longrightarrow$ contingency problem
  input from sensors provide **new** information about current state
  solution is a contingent plan or a policy
  often **interleave** search, execution.
- Unknown state space $\Longrightarrow$ exploration problem ("online").

Example: Vacuum World

Single-state, start in #5. Solution??

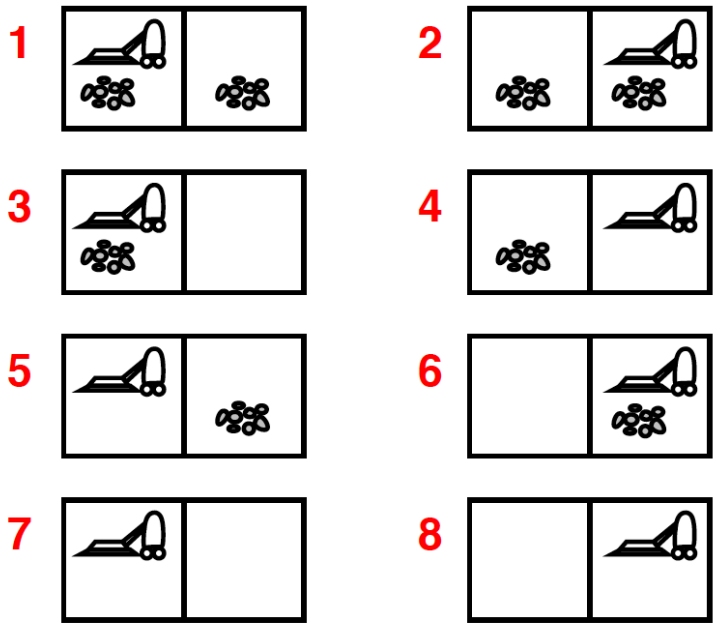


Assume actions Left, Right, Suck (up the dirt), or NoOp (do nothing).

Example: Vacuum World

Single-state, start in #5. Solution??
[*Right*, *Suck*]

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$
e.g., *Right* goes to $\{2, 4, 6, 8\}$. Solution??

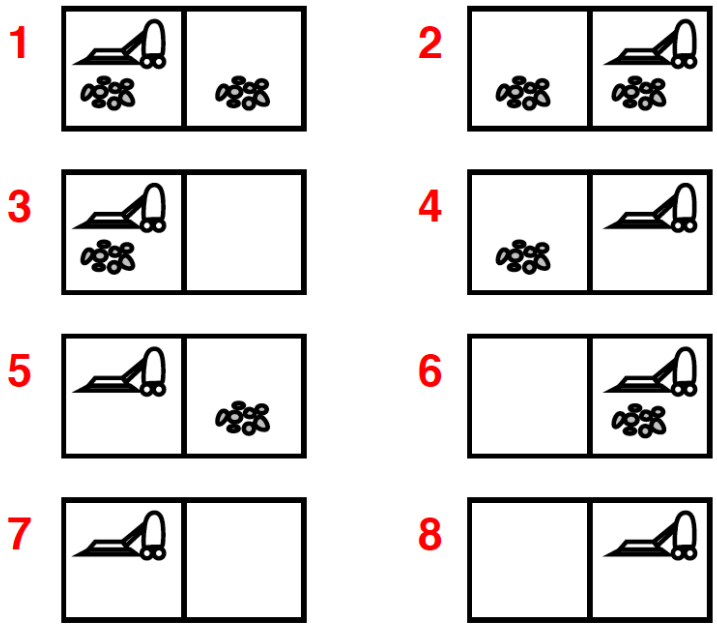


Example: Vacuum World

Single-state, start in #5. Solution??
[*Right*, *Suck*]

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$
e.g., *Right* goes to $\{2, 4, 6, 8\}$. Solution??
[*Right*, *Suck*, *Left*, *Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??

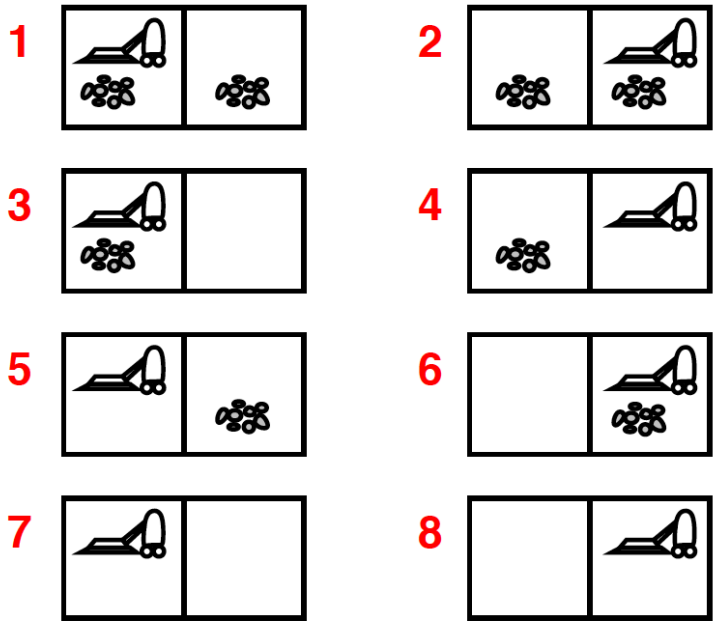


Example: Vacuum World

Single-state, start in #5. Solution??
[*Right*, *Suck*]

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$
e.g., *Right* goes to $\{2, 4, 6, 8\}$. Solution??
[*Right*, *Suck*, *Left*, *Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??
[*Right*, *if dirt then Suck*]



Problems solved by (classical) search

The environment is:

- observable
  - current state known;
- discrete
  - from each state finitely many next states (by executing actions);
- deterministic
  - each action has exactly one outcome (next state);
- known
  - possible actions and next states for each state.

Outline

1. Solving problems by search
2. Problem types
3. Problem formulation
4. Example problems

Single-state problem formulation

A problem description is defined by six items:

1. set of states e.g. $\{At(Arad), At(Zerind), ..., At(Eforie))\}$.
2. initial state e.g., $At(Arad)$.
3. a set of actions e.g. $\{DriveTo(city)\}$.
4. Transition model possibly via a successor function
   $RESULT : States \times Action \rightarrow State$
   e.g., $RESULT(At(Arad), DriveTo(Zerid)) = At(Zerid)$.
5. goal test, can be
   explicit e.g.,$At(Bucharest)$ or implicit e.g., $\forall x \neg Dirt(x)$.
6. path cost (additive e.g. sum of positive step costs), where $c(x, a, y)$ is the step cost of action a, assumed to be ≥ 0. E.g. sum of distances, number of actions, etc.

A solution is a path (seq. of actions) from the initial state to a goal state. Optimal solution is solutions with the lowest path cost.

Selecting a state space

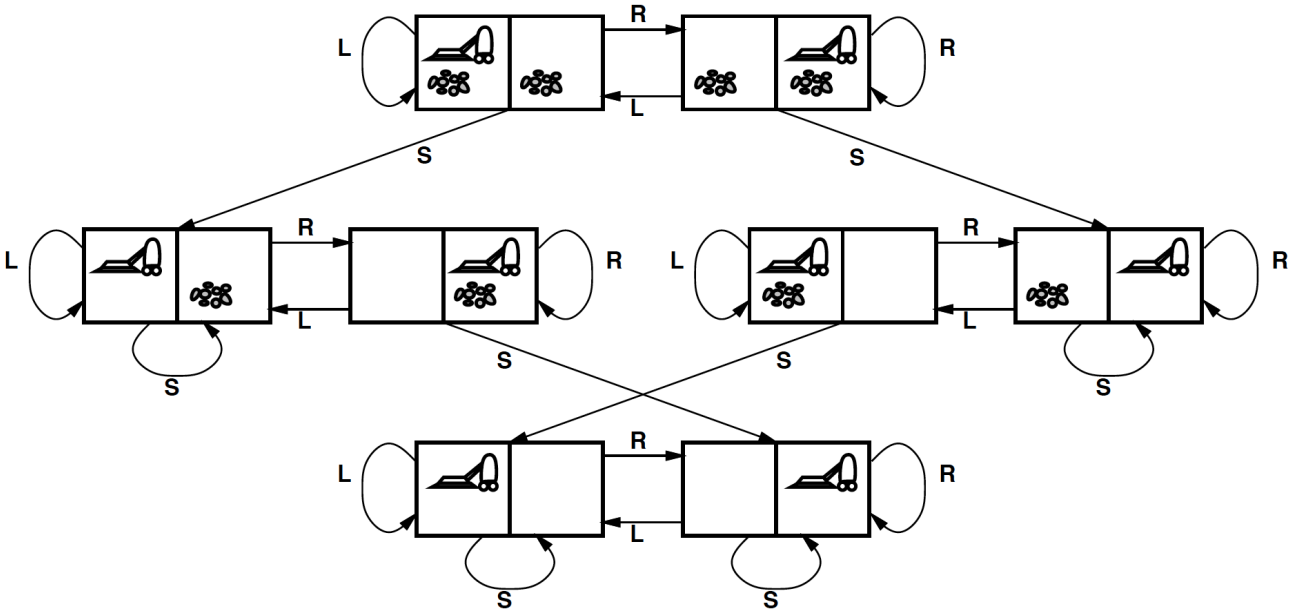
- Real world is absurdly complex
  $\Rightarrow$ state space must be **abstracted** for problem solving.
- (Abstract) state = set of real states
- (Abstract) action = complex combination of real actions
  e.g., $RESULT(At(Arad), DriveTo(Zerid)) = At(Zerid)$ represents a complex set of possible routes, detours, rest stops, etc.
  For guaranteed realizability, **any** real state $At(Arad)$ must get to some real state $At(Zerid)$.
- (Abstract) solution =
  set of real paths that are solutions in the real world

Each abstract action should be "easier" than the original problem!

Outline

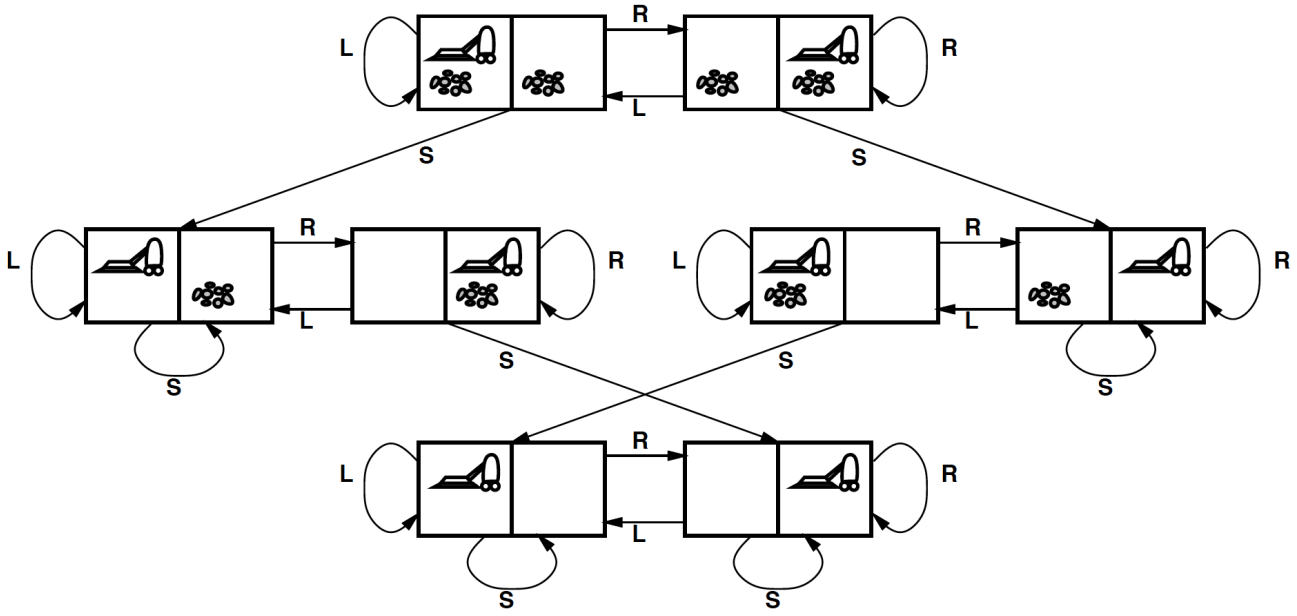
1. Solving problems by search
2. Problem types
3. Problem formulation
4. Example problems

Example: vacuum world state space graph



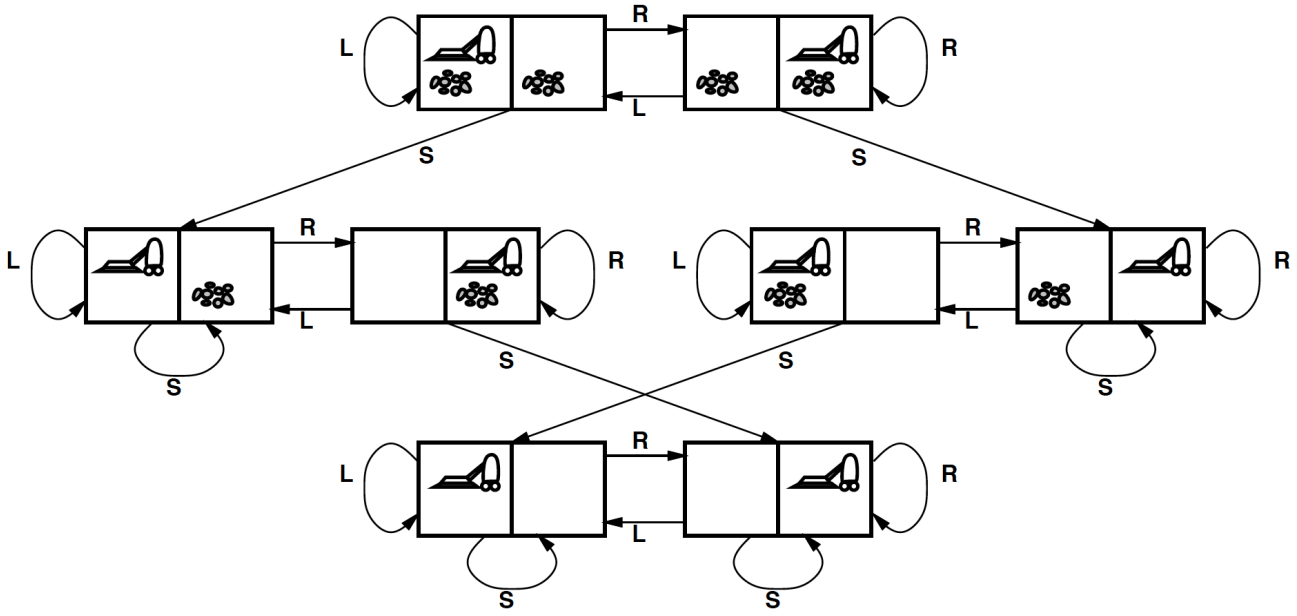
states??
initial state??
actions??
transition model??
goal test??
path cost??

Example: vacuum world state space graph



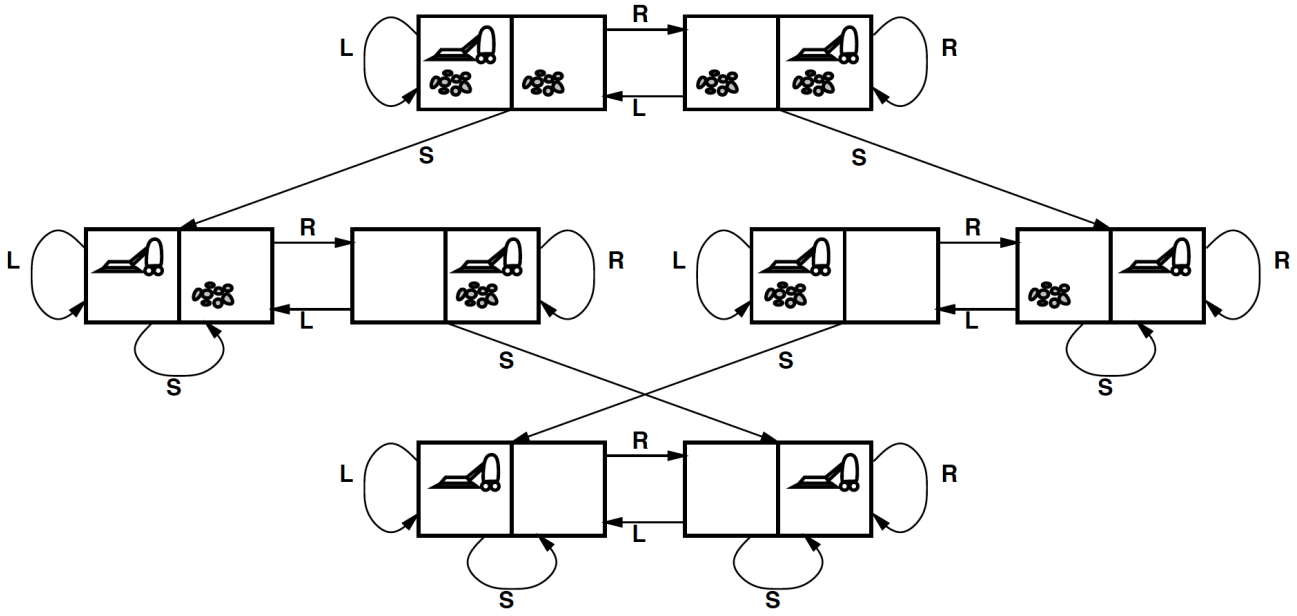
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state??
actions??
transition model??
goal test??
path cost??

Example: vacuum world state space graph



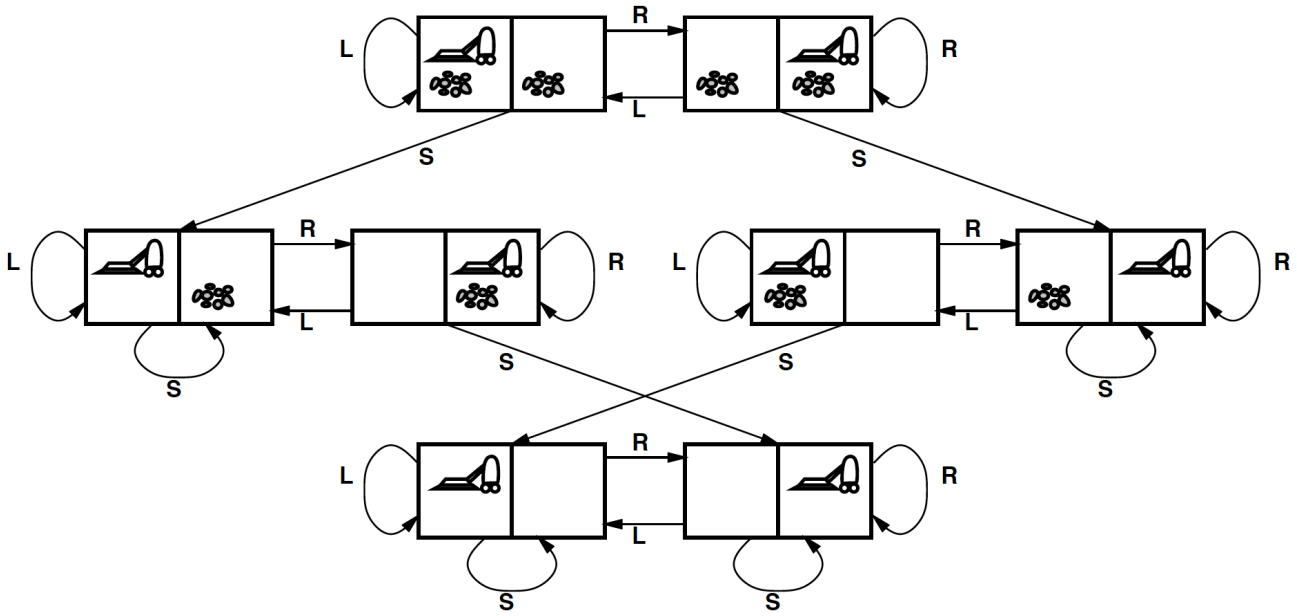
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state?? e.g. $In(A) \wedge Dirt(A) \wedge Dirt(B)$
actions??
transition model??
goal test??
path cost??

Example: vacuum world state space graph



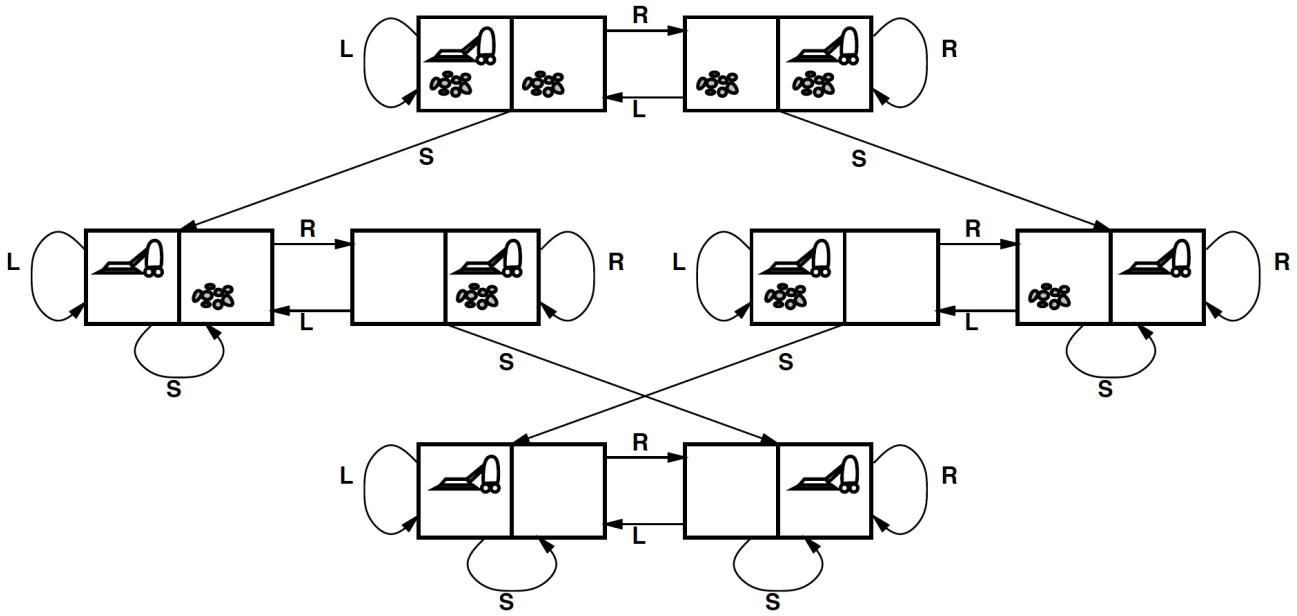
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state?? e.g. $In(A) \wedge Dirt(A) \wedge Dirt(B)$
actions?? $Left, Right, Suck, NoOp$.
transition model??
goal test??
path cost??

Example: vacuum world state space graph



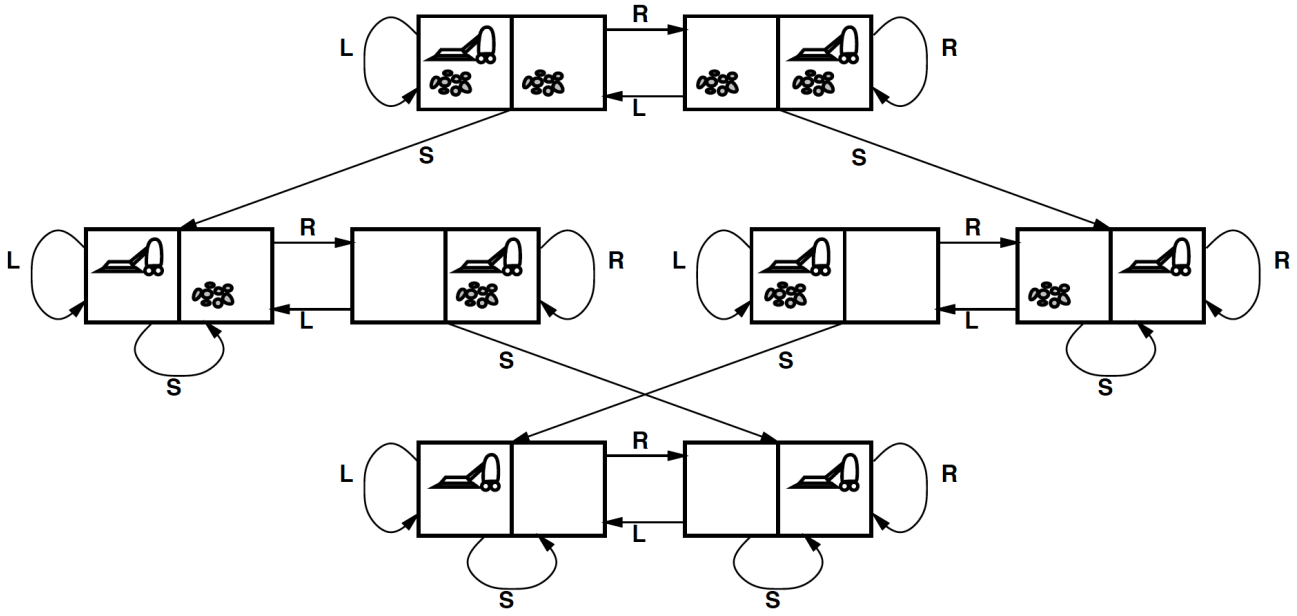
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state?? e.g. $In(A) \wedge Dirt(A) \wedge Dirt(B)$
actions?? $Left, Right, Suck, NoOp$.
transition model?? $RESULT(In(A) \wedge Dirt(A) \wedge Dirt(B), Suck) = In(A) \wedge \neg Dirt(A) \wedge Dirt(B))$ – one of many.
goal test??
path cost??

Example: vacuum world state space graph



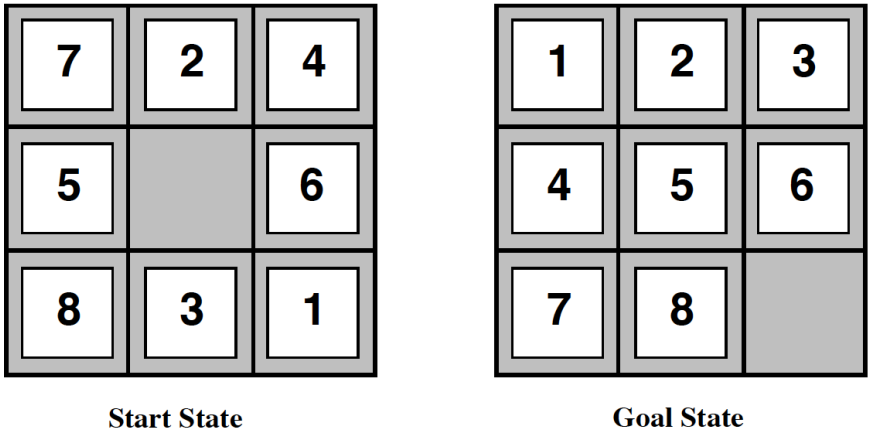
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state?? e.g. $In(A) \wedge Dirt(A) \wedge Dirt(B)$
actions?? $Left, Right, Suck, NoOp$.
transition model?? $RESULT(In(A) \wedge Dirt(A) \wedge Dirt(B), Suck) = In(A) \wedge \neg Dirt(A) \wedge Dirt(B))$ – one of many.
goal test?? $\forall x \neg Dirt(x)$
path cost??

Example: vacuum world state space graph



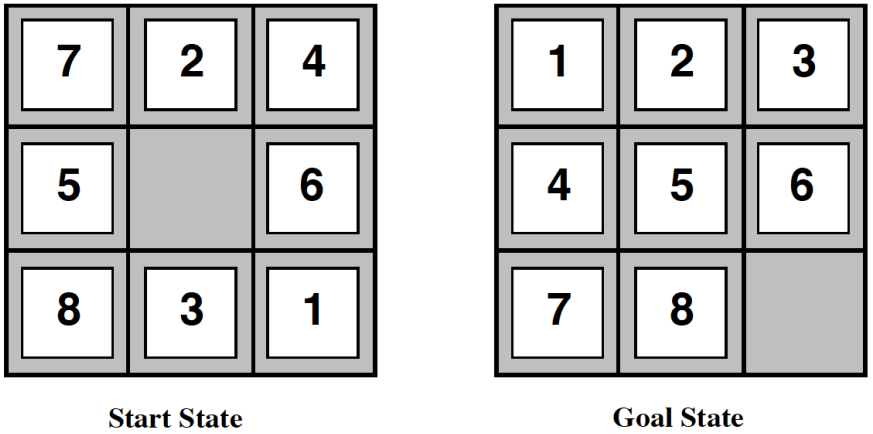
states?? For n locations $n \times 2^n$ possible states. For $n = 2$, then 8 states.
initial state?? e.g. $In(A) \wedge Dirt(A) \wedge Dirt(B)$
actions?? $Left, Right, Suck, NoOp$.
transition model?? $RESULT(In(A) \wedge Dirt(A) \wedge Dirt(B), Suck) = In(A) \wedge \neg Dirt(A) \wedge Dirt(B))$ – one of many.
goal test?? $\forall x \neg Dirt(x)$
path cost?? 1 per action (0 for $NoOp$)

Example: The 8-puzzle



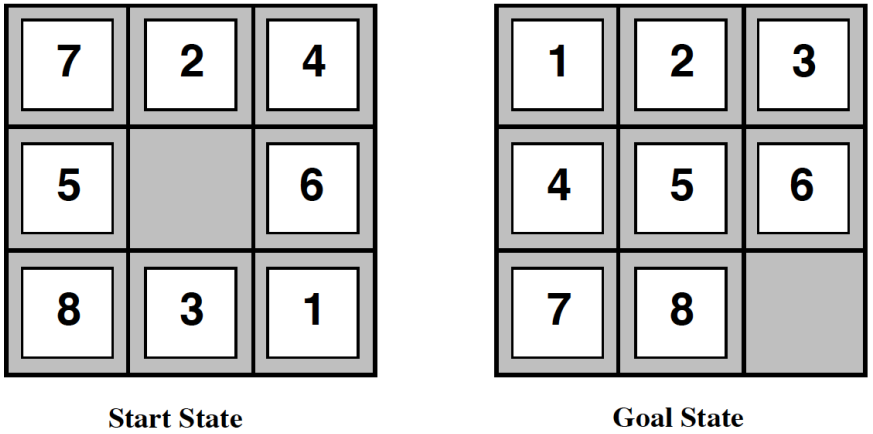
states??
initial state??
actions??
transition model??
goal test??
path cost??

Example: The 8-puzzle



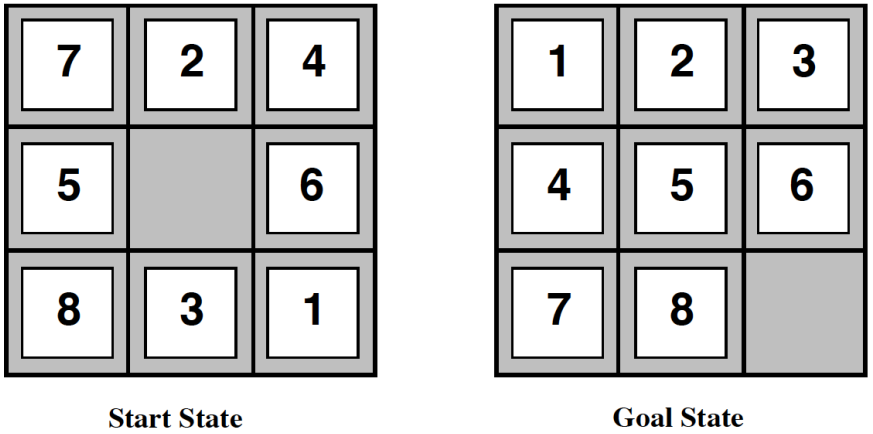
states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state??
actions??
transition model??
goal test??
path cost??

Example: The 8-puzzle



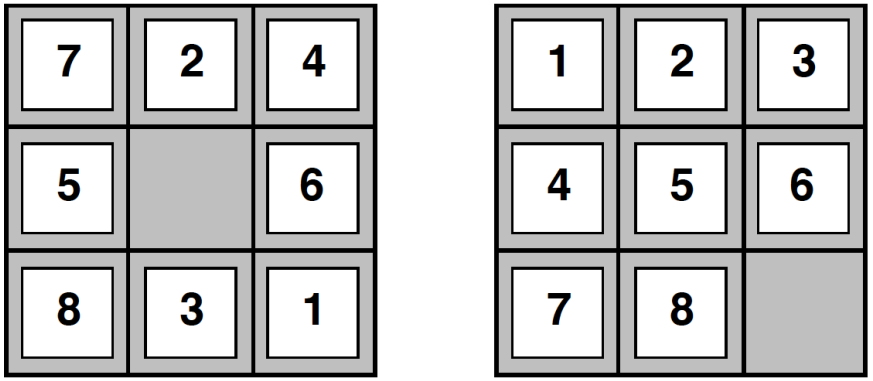
states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state?? e.g. $Tile(1,1,7) \wedge Tile(1,2,2) \cdots \wedge Empty(2,2)$.
actions??
transition model??
goal test??
path cost??

Example: The 8-puzzle



states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state?? e.g. $Tile(1,1,7) \wedge Tile(1,2,2) \cdots \wedge Empty(2,2)$.
actions?? *Up*, *Down*, *Left*, *Right*.
transition model??
goal test??
path cost??

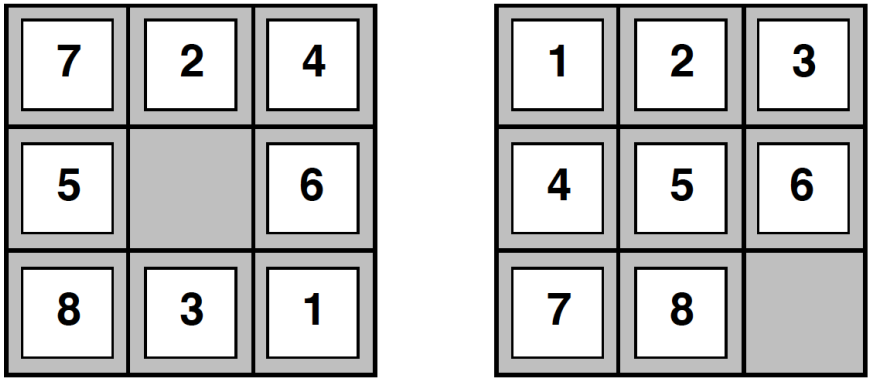
Example: The 8-puzzle



Start State Goal State

states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state?? e.g. $Tile(1,1,7) \wedge Tile(1,2,2) \cdots \wedge Empty(2,2)$.
actions?? Up, $Down$, $Left$, $Right$.
transition model?? $RESULT(Tile(1,1,7) \wedge \cdots \wedge Empty(2,2), Up) = Tile(1,1,7) \wedge \cdots \wedge Empty(1,2) \cdots +$ *more transitions*
goal test??
path cost??

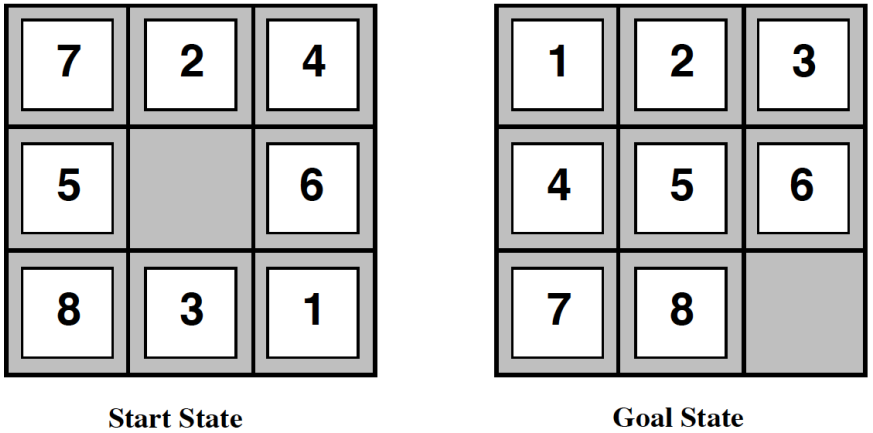
Example: The 8-puzzle



Start State Goal State

states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state?? e.g. $Tile(1,1,7) \wedge Tile(1,2,2) \cdots \wedge Empty(2,2)$.
actions?? Up, $Down$, $Left$, $Right$.
transition model?? $RESULT(Tile(1,1,7) \wedge \cdots \wedge Empty(2,2), Up) =$
$Tile(1,1,7) \wedge \cdots \wedge Empty(1,2) \cdots +$ *more transitions*
goal test?? $Tile(1,1,1) \cdots \wedge Tile(2,2,5) \cdots \wedge Empty(3,3)$
path cost??

Example: The 8-puzzle



states?? The 8-puzzle has $9!/2$=181, 440 possible states.
initial state?? e.g. $Tile(1,1,7) \wedge Tile(1,2,2) \cdots \wedge Empty(2,2)$.
actions?? Up, $Down$, $Left$, $Right$.
transition model?? $RESULT(Tile(1,1,7) \wedge \cdots \wedge Empty(2,2), Up) = Tile(1,1,7) \wedge \cdots \wedge Empty(1,2) \cdots +$ *more transitions*
goal test?? $Tile(1,1,1) \cdots \wedge Tile(2,2,5) \cdots \wedge Empty(3,3)$
path cost?? 1 per move

Note: optimal solution of n-Puzzle family is NP-hard.

Summary

Formulation of a search problem usually requires:

- abstracting away real-world details as:
  1. a set of states;
  2. the initial state;
  3. a set of actions;
  4. a transition model;
  5. the goal test;
  6. a path-cost function;
- specifying the state space so that it can be feasibly explored.

Different types of problems exist. We will be concerned with deterministic and fully observable environments i.e. **single-state problems**.